
DOCUMENTO TÉCNICO — PIPELINE CI/CD

Sistema de Gestión de Eventos

Rama GitFlow	feature/cd-documentation
Plataforma CI/CD	GitHub Actions
Backend / Frontend	Java 21 + Spring Boot / React + Node.js 20
Ambiente staging	Railway (PaaS)
Análisis de seguridad	Semgrep (SAST)

1. Flujo General del Pipeline

El pipeline está compuesto por cinco workflows de GitHub Actions con responsabilidades separadas:

Archivo	Trigger	Propósito
ci.yml	push/PR → develop, main	Compilación y pruebas unitarias del backend
automated-build.yml	push/PR → develop, main	Generación de artefactos JAR (backend) y bundle (frontend)
cd.yml	push/PR → develop, main	Construcción de imagen Docker
cd-staging.yml	push → develop	Despliegue en Railway + health check post-despliegue
semgrep-security-scan.yml	push/PR → develop, main	Análisis estático de seguridad (SAST)

2. Herramientas Utilizadas

Herramienta	Rol
GitHub Actions	Orquestación de todos los pipelines
Java 21 / Maven	Compilación, pruebas y empaquetado del backend Spring Boot
Node.js 20 / npm	Compilación del frontend React
Docker	Construcción de imagen OCI del backend
Semgrep	Análisis estático de seguridad (reglas auto + bloqueo en main)
Railway	Plataforma PaaS para despliegue automático del staging
Spring Actuator	Endpoint /actuator/health para verificación post-despliegue

3. Etapas del Pipeline

3.1 Integración Continua (ci.yml)

- Configura JDK 21 con caché Maven.
- Compila con mvn clean compile y ejecuta pruebas con mvn test -Dspring.profiles.active=test.
- Variables SMTP inyectadas desde GitHub Secrets. Si las pruebas fallan, el pipeline se detiene.

3.2 Build Automatizado (automated-build.yml)

- Job build-backend: genera el JAR con mvn clean package -DskipTests; lo sube como artefacto (7 días).
- Job build-frontend: instala dependencias con npm ci, compila con npm run build; sube dist/ como artefacto.
- Job build-summary: se ejecuta solo si ambos builds son exitosos.

3.3 CD Pipeline (cd.yml)

- Job build: empaqueta el backend con Maven.
 - Job docker (needs: build): construye la imagen proyectodevops-backend:latest con el Dockerfile.
 - Job deploy-info (needs: docker): imprime metadata del despliegue (rama, commit, actor).
-

3.4 Despliegue a Staging (cd-staging.yml)

- Job build: mvn clean test package (incluye pruebas); persiste el JAR 3 días.
- Job deploy-staging: Railway detecta el push a develop y despliega automáticamente; el job espera 90 s.
- Job verify-deployment: 5 intentos de curl al endpoint /actuador/health (timeout 10 s, pausa 20 s entre intentos). HTTP 200 = éxito; si fallan los 5, exit 1 marca el despliegue como fallido.

4. Análisis de Seguridad con Semgrep

Semgrep examina archivos `**/*.java`, `**/*.yml` y `**/*.properties` usando `--config=auto` (reglas oficiales).

Job	Rama	continue-on-error	Efecto ante hallazgos
semgrep-scan	develop + main + PRs	true	Genera reporte JSON como artefacto; no bloquea
semgrep-block	main únicamente	false	Bloquea el pipeline ante severidad ERROR

El token de autenticación (SEMGREP_APP_TOKEN) se gestiona como GitHub Secret y nunca aparece en el repositorio.

5. Gestión de Variables y Secretos

Variable / Secret	Pipeline	Propósito
SMTP_HOST / PORT / USER / PASS	ci.yml	Configuración del servidor de correo para pruebas
SEMGREP_APP_TOKEN	semgrep-security-scan.yml	Autenticación con la plataforma Semgrep
STAGING_URL (vars.*)	cd-staging.yml	URL base del ambiente Railway para el health check

6. Comportamiento ante Fallas

Punto de falla	Consecuencia
Pruebas unitarias (ci.yml)	Pipeline detenido; el código no avanza a ninguna etapa posterior
Semgrep-block en main	Merge bloqueado; vulnerabilidad crítica no llega a producción
Health check post-despliegue	Pipeline marcado como fallido tras 5 intentos; despliegue roto queda visible
Build Docker fallido	Jobs dependientes no se ejecutan (needs: docker)

7. Decisiones Técnicas Clave

- Railway como staging: despliegue automático sin gestión de infraestructura; tier gratuito suficiente.
 - Jobs separados por responsabilidad: facilita identificar exactamente qué etapa falló.
 - Semgrep `--config=auto`: cubre patrones comunes Java/Spring sin configuración adicional.
 - sleep 90 antes del health check: Railway necesita ese tiempo para construir y levantar el contenedor.
 - Artefactos con retención 3–7 días: balance entre almacenamiento y disponibilidad para auditoría.
-

8. Entregables

- Repositorio con los 5 archivos de pipeline en `.github/workflows/`.
- Evidencia de ejecución exitosa (capturas de GitHub Actions).
- Evidencia del ambiente staging operativo (URL Railway).
- Este documento técnico.
- Evidencia de falla controlada (Semgrep bloqueando o health check fallido).